

```
def greet(name)
  "Hello, #{name}!"
end

puts greet("Alice")
```

Classes and Objects:

Ruby is an object-oriented language, meaning it revolves around objects and classes.

```
# Define a class
class Person
  attr_accessor :name, :age

  def initialize(name, age)
    @name = name
    @age = age
  end

  def display
    puts "Name: #{@name}, Age: #{@age}"
  end
end

# Create an object
person = Person.new("Alice", 30)
person.display
```

Writing Your First Ruby Program

Let's write a simple Ruby program that asks the user for their name and age, then displays a greeting message and calculates the year they were born.

1. Create a New Ruby File:

- Open your text editor or IDE and create a new file named `greeting.rb`.

2. Write the Ruby Code:

- Enter the following code in the `greeting.rb` file:

```
# Ask for the user's name
print "Enter your name: "
name = gets.chomp

# Ask for the user's age
print "Enter your age: "
age = gets.to_i

# Get the current year
current_year = Time.now.year

# Calculate the year of birth
birth_year = current_year - age
```

```
# Display the result
puts "Hello, #{name}! You were born in
#{birth_year}."
```

3. Run the Program:

- Open a terminal and navigate to the directory containing the `greeting.rb` file.
- Run the program by executing `ruby greeting.rb`.
- Enter your name and age when prompted and verify the output.

Advanced Ruby Features

Ruby offers a range of advanced features that enable developers to write efficient and elegant code. Some of these features include modules, blocks, procs, lambdas, and metaprogramming.

Modules:

Modules are a way to group methods, classes, and constants. They can be used to mix in behavior to classes.

```
module Greetings
  def greet(name)
    "Hello, #{name}!"
  end
end

class Person
  include Greetings
end

person = Person.new
puts person.greet("Alice")
```

Blocks:

Blocks are chunks of code enclosed in `do...end` or `{...}` and can be passed to methods.

```
3.times do |i|
  puts "Iteration: #{i}"
end

[1, 2, 3].each { |n| puts n * 2 }
```

Procs and Lambdas:

Procs and lambdas are objects that store blocks of code to be executed later.

```
# Proc
my_proc = Proc.new { |name| "Hello,
#{name}!" }
puts my_proc.call("Alice")

# Lambda
my_lambda = -> (name) { "Hello, #{name}!" }
puts my_lambda.call("Alice")
```